Project Vela — Workflow Guide

v0.6.0 • Windows 11 offline media player • multi-AI workflow

What this PDF is: a step-by-step workflow that minimizes errors when you use Cursor + Google Jules + ChatGPT 5.2 Pro + Gemini Ultra + GitHub.

Source type: This workflow is based on *common industry practice* for PR-based development, plus the *official docs* of each tool, plus your *project requirements* captured in docs/SPEC.md.

Quick summary of what we're building (so we don't drift)

Project Vela is a Windows 11, fully offline, operator-driven audio/video playback app for conferences and live events.
Key constraints: never crash, two-window model (control + output), folder watching, drag & drop, playlists, hotkeys (only when focused), and a panic blackout feature.

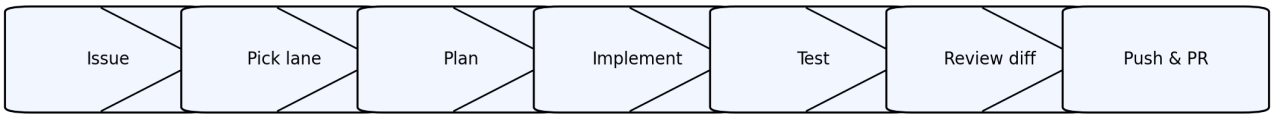
Source type: User requirements + repo spec (docs/SPEC.md).

The Golden Loop

This loop is your "error minimizer". Every task follows the same shape so you don't accidentally create AI chaos.

Source type: Common industry practice.

Project Vela — Golden Loop (One Issue → One PR)



Tool roles (who does what)

Keep each tool in its lane. This prevents duplicated edits and reduces merge conflicts.

Tool	Best for	Hard rules (error-proofing)

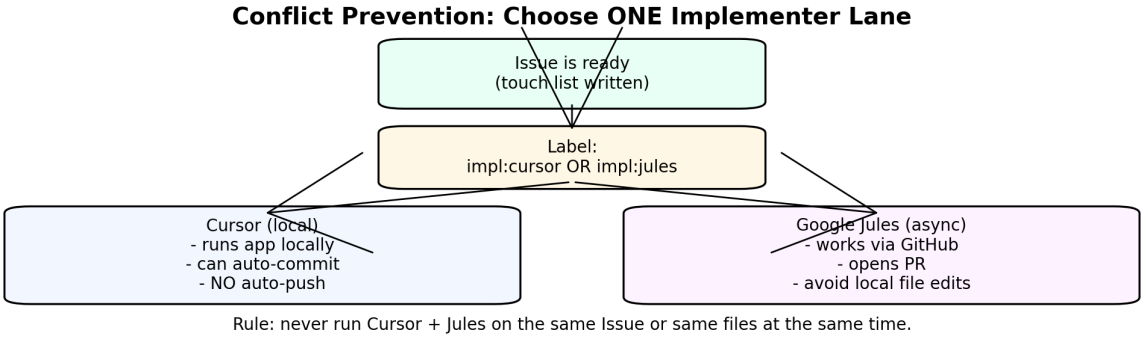
Cursor	Local implementation + debugging with terminal access	One Issue at a time; auto-commit OK; **no auto-push**; follow .cursor/rules.
Google Jules	Async PRs from clearly-scoped Issues	Run only on Issues labeled impl:jules; avoid overlapping files with Cursor.
ChatGPT 5.2 Pro	Planning, specs, PR review, writing Issues	Don’t write code straight to main; always produce a touch list + test plan.
Gemini Ultra (Deep Think)	Red-team planning; edge cases; performance/stability checklists	Use it to attack plans and find missing failure modes.
GitHub	Single source of truth for Issues, PRs, and CI	Protect main; require CI green; one PR per Issue.

Source types: Official developer documentation for Cursor/Jules/GitHub/OpenAI/Google; common industry practice for lane separation and PR workflow.

Prevent Cursor ↔ Jules conflicts

v0.6.0

You asked: “Cursor can commit automatically—how do I prevent conflicts with Jules editing the same file at the same time?” This section is the answer.



Rules that stop merge conflicts before they happen

- **Lane label:** every Issue has exactly one: *impl:cursor* OR *impl:jules*.
- **Touch list:** the Issue must list the files that are allowed to change.
- **Single writer:** only one agent edits those files until the PR merges.
- **Auto-commit, manual push:** Cursor can commit; you push after you review the diff.

Source type: Common industry practice + your workflow preference.

Setup checklist (do once)

GitHub

- Create repo from this starter (or unzip and push).
- Enable branch protection for **main**: require PR + require CI green.
- Create labels: impl:cursor, impl:jules, jules, scope:docs, scope:code.

Source type: GitHub official documentation + repo conventions.

Cursor

- Open the repo folder in Cursor.
- Verify .cursor/rules exists and is being applied.
- Enable agent terminal mode only if you’re comfortable reviewing what it runs.

Source type: Cursor official documentation + common practice.

Google Jules

- Install/authorize the Jules GitHub App for this repo.
- Ensure AGENTS.md exists at repo root (Jules reads it).
- Start Jules tasks from Issues labeled **jules** (case-insensitive).

Source type: Jules official documentation.

ChatGPT 5.2 Pro

- Create a dedicated ChatGPT Project named “Project Vela”.
- Upload: docs/SPEC.md, docs/ARCHITECTURE.md, docs/WORKFLOW_AI.md, docs/PROMPTS.md.
- Paste the Universal Header from docs/PROMPTS.md into your Project instructions.

Source type: OpenAI official documentation + this repo.

Gemini Ultra

- Use Gemini as a “red-team planner”: attack plans and find missing failure modes.
- Keep outputs action-oriented: edge cases + test steps.

Source type: Common industry practice.

Daily workflow (repeat for every Issue)

This is the “idiot-proof” checklist you run every time.

Step 1 — Create / pick the Issue

- Write Goal + Non-goals + Defaults/Toggles + Acceptance criteria.
- Add a touch list (files allowed to change).

Step 2 — Choose the implementer lane

- If you need local debugging → choose impl:cursor.
- If it’s a well-scoped change → choose impl:jules.
- Never run both lanes at once.

Step 3 — Plan (ChatGPT)

- Ask for: plan (≤10 steps), touch list, risk list, verification steps.
- If ChatGPT is uncertain, it must say so and ask for missing files/logs.

Step 4 — Red-team (Gemini)

- Ask Gemini to attack the plan for crash risks and edge cases.
- Add missing acceptance checks to the Issue before implementing.

Step 5 — Implement (Cursor OR Jules)

- Cursor: local dev + tests + auto-commit; STOP; no push.
- Jules: start from Issue; review plan; let it open PR.

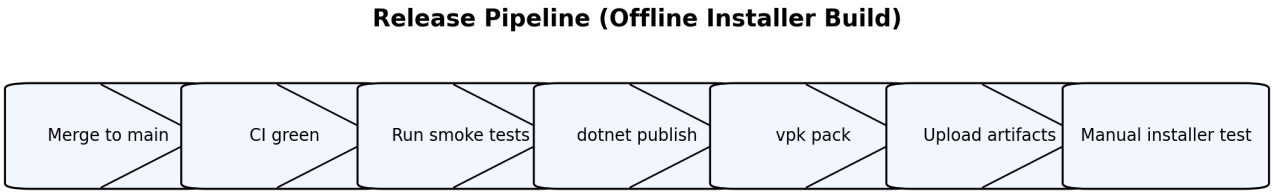
Step 6 — Verify (Human)

- Run dotnet test and smoke test steps.
- Glance at git diff: does it match the Issue and touch list?
- Push (if Cursor lane) and open PR.

Step 7 — Merge & package

- Merge only when CI is green.
- Tag a release and run the Velopack workflow when ready.

Release pipeline diagram



Source type: Repo automation (GitHub Actions workflows) + Velopack official documentation.

The 6 highest-leverage prompts

Use these to keep AIs on rails.

Prompt	Where
ChatGPT — Lane decision + touch list	docs/PROMPTS.md → 1.2
ChatGPT — Beginner test plan	docs/PROMPTS.md → 1.4
Gemini — Red-team a plan	docs/PROMPTS.md → 4.1
Cursor — Implement Issue (commit/no push)	docs/PROMPTS.md → 2.1
Jules — Implement Issue (small PR)	docs/PROMPTS.md → 3.1
ChatGPT — PR review	docs/PROMPTS.md → 1.5

Sources & official docs (clickable)

Item	Link	Source type
Cursor Docs — Rules	https://cursor.com/docs/context/rules	Official developer documentation
Cursor Docs — Agent terminal	https://cursor.com/docs/agent/terminal	Official developer documentation
Cursor Security	https://cursor.com/security	Official developer documentation
Jules Docs — Getting started	https://jules.google/docs/	Official developer documentation
Jules Docs — Running tasks from GitHub Issues (label “jules”)	https://jules.google/docs/running-tasks/	Official developer documentation
GitHub Docs — Protected branches	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches	Official developer documentation
Velopack Docs — Packaging overview	https://docs.velopack.io/packaging/overview	Official developer documentation

Velopack Docs — Windows overview (install location, no privileged dirs)	https://docs.velopack.io/packaging/operating-systems/windows	Official developer documentation
OpenAI Help — ChatGPT capabilities overview (Projects)	https://help.openai.com/en/articles/9260256-chatgpt-capabilities-overview	Official developer documentation
OpenAI Help — File Uploads FAQ	https://help.openai.com/en/articles/8555545-file-uploads-faq	Official developer documentation
Google AI Dev — Gemini thinking	https://ai.google.dev/gemini-api/docs/thinking	Official developer documentation